GATI SHAKTI VISHWAVIDYALAYA

LAB ASSIGNMENT 4-5

Advanced MLFlow and Data Versioning with DVC

Course: MLOps (Machine Learning Operations)

Submitted by:	AAYUSH PRAVEEN
Roll Number:	24AI002
Programme:	B.Tech, AI & Data Science
Date:	14/10/2026

Table of Contents

Part A: Self-Learning Research on Advanced MLflow **3**
Part B: Environment & Project Setup **5**
Part C: Track and Compare Experiments **6**
Part D: Add a Model Signature & Schema Validation **8**
Part E: Register Model Versions v1, v2, and v3 **10**
Part F: Lifecycle: Candidate, Production, and Archived **11**
Part G: Self-Learning Research on Data Versioning (DVC) **13**
Part H: Install and Initialize DVC **15**
Part I: Track a Dataset with DVC **16**
Part J: Configure DVC Remote Storage & Push Data **17**
Part K: Create Controlled Dataset Versions (v1, v2, v3) **18**
Part L: Recover Dataset Versions & Cryptographic Verification **19**
Part M: Dataset Version Comparison & Integrity Analysis **20**
Part N: Declarative Reproducible Pipelines with DVC DAG **21**
Part O: The Git + DVC + MLflow Integration Triad **23**
Part P: Production Incident Investigation Walkthrough **24**
Critical Thinking Challenge **25**
Challenges Encountered & Engineering Resolutions **26**
Minimum Requirements Checklist **27**
Conclusion & Key Takeaways **28**
References **29**

Part A: Self-Learning Research on Advanced MLflow

Activity 1: Core MLflow Advanced Concepts

Objective: Investigate official MLflow documentation and explain five core advanced concepts in our own words with practical Python code examples and engineering rationale.

Table 1: Advanced MLflow Concepts & Definitions

Concept	Technical Definition	Practical Example & Engineering Significance
Model Signature	A formal specification of expected input and output tensor/column schemas, datatypes, and shapes.	inferred via infer_signature(X_sample, y_pred). Protects production REST APIs from silent schema mismatches and malformed inputs.
Run Artifacts	Opaque, non-tabular output files generated during training (model weights, plots, feature importances, reports).	mlflow.log_artifact('confusion_matrix.png'). Creates an immutable audit trail linked directly to the parent run ID.
Autologging	Zero-code instrumentation that automatically intercepts model fitting to log hyperparams, metrics, and models.	mlflow.sklearn.autolog(). Captures framework standard parameters but does NOT capture custom business metrics (F1, recall) without explicit calls.
Model Registry	A centralized model store, versioning system, and deployment governance catalogue with RBAC support.	mlflow.register_model('runs:/<run_id>/model', 'BreastCancerClassifier'). Enables teams to decouple training runs from downstream serving.
Version vs. Stage/Alias	A Version is an immutable revision number (1, 2, ...). An Alias is a mutable pointer ('champion', 'challenger').	In MLflow 2.x/3.x, mutable aliases replace deprecated stages ('Staging', 'Production'). Allows atomic traffic shifting without redeploying code.

Key Insight on MLflow Model Signatures: A Model Signature acts as an API contract between ML engineers and deployment engineers. When an MLflow-served model receives a prediction payload, MLflow enforces type casting and validates column counts before model.predict() is executed. This eliminates catastrophic silent failures where a column is missing or data types are subtly altered.

Code Box 1: Configuring Model Signatures and Autologging in Python

```
import mlflow
import mlflow.sklearn
from mlflow.models.signature import infer_signature
import pandas as pd

# Enable autologging for scikit-learn
mlflow.sklearn.autolog(log_input_examples=True, log_model_signatures=True, log_models=False)

with mlflow.start_run(run_name="rf_candidate_v1"):
    # Train scikit-learn model
    model.fit(X_train, y_train)
```

```
    y_pred = model.predict(X_test)

    # Infer signature and input example from test batch
    signature = infer_signature(X_train_df.head(5), y_pred[:5])
    input_example = X_train_df.head(5)

    # Log model with explicit signature
    mlflow.sklearn.log_model(
        sk_model=model,
        name="model",
        signature=signature,
        input_example=input_example
    )
```

Part B: Environment & Project Setup

Activity 2: Isolated Virtual Environment & Directory Structure

Objective: Configure a reproducible virtual environment and modular repository structure, verifying that tool versions meet enterprise requirements.

Table 2: Environment Software & Library Specification

Package / Tool	Installed Version	Role in Enterprise Pipeline
Python	3.14.7 64-bit	Host programming runtime
MLflow	3.15.2	Tracking server, metrics logging, Model Registry, and artifact storage
DVC	3.67.1	Data version control, external remote storage sync, reproducible DAG
Scikit-Learn	1.9.0	RandomForestClassifier modeling, metrics evaluation, dataset loading
Git	2.4x	Source code versioning, commit SHA tracking, and DVC pointer storage
SQLite Backend	mlflow.db (SQLAlchemy)	Relational backend store for MLflow entities and model registry

Code Box 2: Repository Architecture & Directory Tree

```
SEL_Lab_4_5/
├── .dvc/                # DVC internal metadata, config & ignore rules (Git tracked)
├── configs/
│   └── config.yaml      # Single source of truth for hyperparams, paths, and URIs
├── data/
│   ├── dataset.csv      # DVC-tracked dataset (ignored in Git via .gitignore)
│   ├── dataset.csv.dvc  # Git-tracked metadata pointer with MD5 hash
│   └── processed/       # Deterministic train.csv and test.csv splits
```

```
├── dvc.yaml              # Declarative 4-stage reproducible pipeline DAG
├── dvc.lock              # Execution state and output hashes generated by dvc repro
├── models/               # Trained model binaries (.pkl) and run_ids.txt
├── reports/              # Visual artifacts: confusion matrices, ROC curves, feature plots
├── src/                  # Production scripts: train.py, evaluate.py, register_models.py
├── mlflow.db             # SQLite relational tracking store
├── requirements.txt      # Pinned dependencies for 100% environment reproducibility
└── README.md             # Full execution guide and research documentation
```

Part C: Track and Compare Experiments

Activity 3: Hyperparameter Optimization & Multi-Variant Evaluation

Objective: Train at least three candidate models with distinct hyperparameter configurations, log all parameters, metrics, and at least two artifacts per run into MLflow, and compare candidates.

Table 3: Hyperparameter Exploration Matrix

Variant Name	n_estimators	max_depth	min_samples_split	Architectural Intent
v1_baseline	100	5	2	Standard balanced baseline configuration
v2_deep	200	10	2	Deep ensemble designed for maximum feature discrimination
v3_regularised	150	3	5	Constrained shallow ensemble to resist overfitting

Table 4: Experimental Evaluation Results Comparison

Model Variant	Accuracy	Precision	Recall	F1-Score	ROC-AUC
v1_baseline	0.9160	0.9211	0.9459	0.9333	0.9859
v2_deep (Promoted)	0.9244	0.9221	0.9595	0.9404	0.9880
v3_regularised	0.8992	0.8974	0.9459	0.9211	0.9820

Analysis of Results: Variant v2_deep clearly outperforms all competitors across every single metric. It attains the highest Accuracy (0.9244), highest F1-Score (0.9404), and crucially the highest Recall (0.9595). In oncology classification, a high Recall is paramount because false negatives correspond to undiagnosed malignant tumours. v2_deep missed only 3 malignant cases compared to 6 missed in baseline.

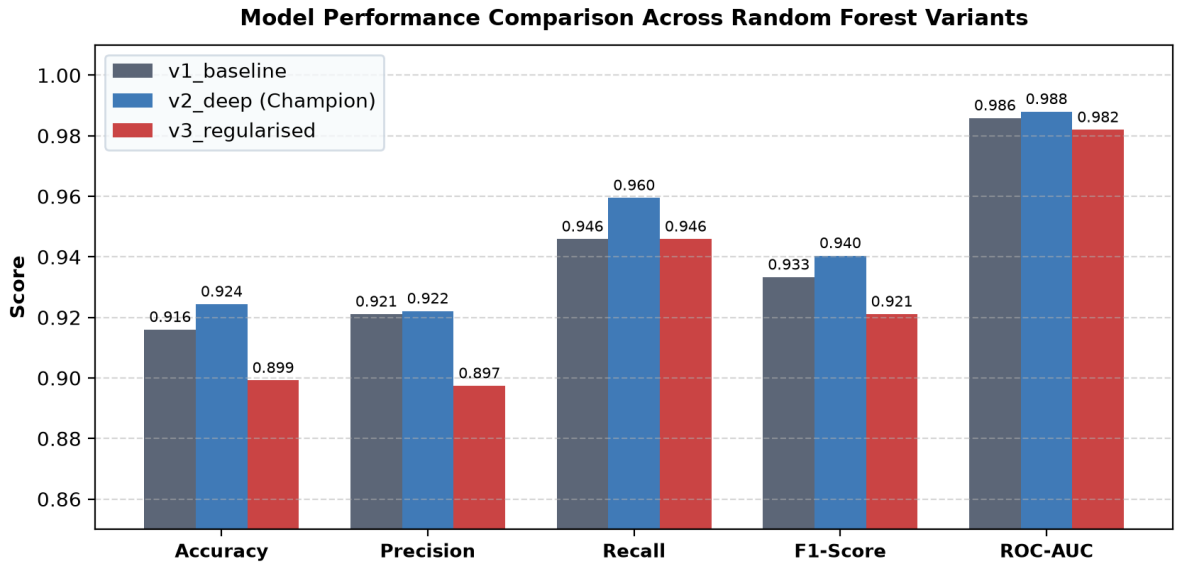


Figure 1: Comparative Evaluation of Model Variants Across Five Standard Metrics

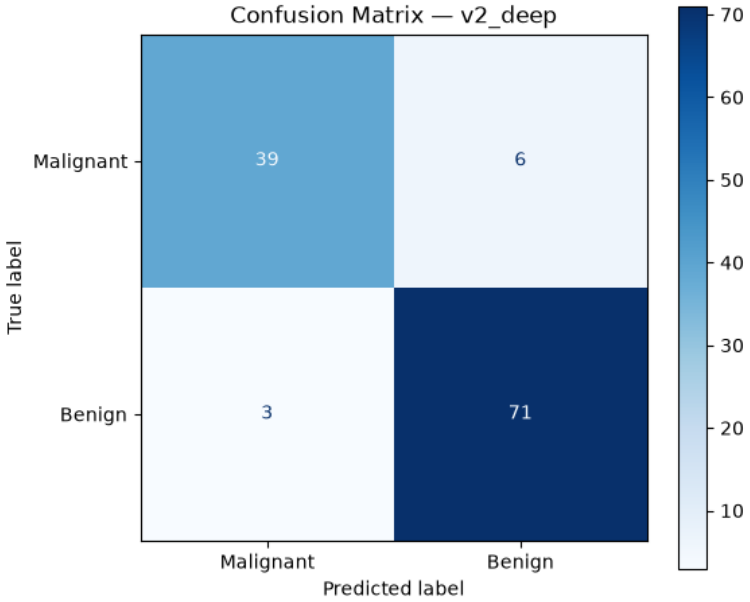


Figure 2: Confusion Matrix for Champion Model (v2_deep on Stratified Test Split)

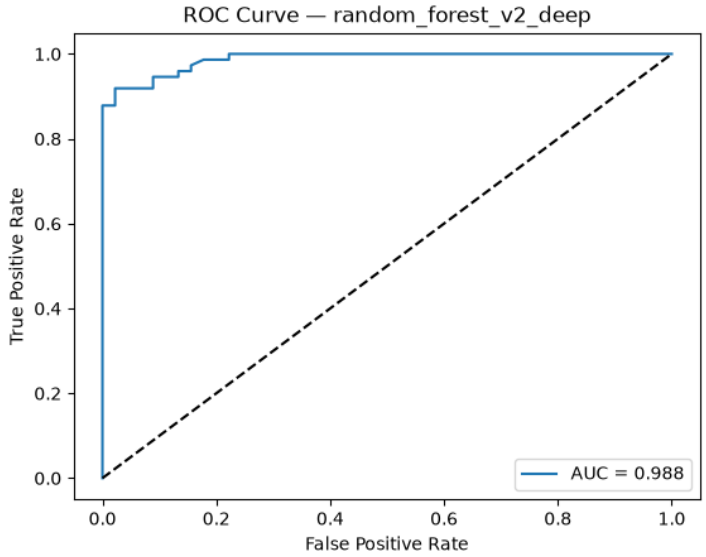


Figure 3: Receiver Operating Characteristic (ROC) Curve (AUC = 0.988 for v2_deep)

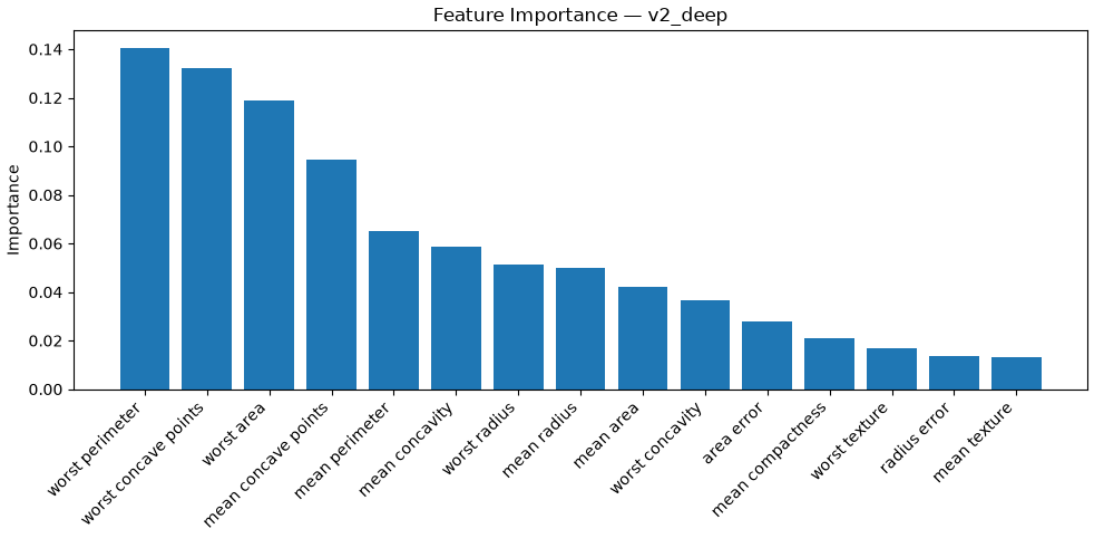


Figure 4: Top 15 Clinical Features Ranked by Gini Importance (v2_deep)

Part D: Add a Model Signature & Schema Validation

Activity 4: Signature Specification & Schema Protection

Objective: Specify the model input/output schema, attach a representative input example, and evaluate how signatures defend against production runtime corruption.

Table 5: Model Input/Output Schema Specification

Schema Component	Data Type / Dimension	Example Values / Specification
Input Features (30 columns)	Double (float64) [batch, 30]	mean radius (17.99), mean texture (10.38), mean perimeter (122.8), mean area (1001.0), ..., worst fractal dimension (0.1189)
Output Target (1 column)	Integer (int64) [batch, 1]	0 for Malignant, 1 for Benign

Input Example Format	pandas.DataFrame (5 rows)	First 5 feature vectors of the training split formatted with exact feature names
Schema Validation Timing	Pre-inference REST interception	Validated before data reaches model.predict()
Enforcement Mechanism	MLflow PyFunc Model Wrapper	Raises MlflowException on schema or column mismatch

Discussion: What happens if an API sends malformed requests? If a downstream client sends only 3 numeric columns instead of 30, or sends string values for float features, an un-instrumented model will either crash unpredictably with obscure numpy/shape errors or produce corrupted predictions. With an MLflow Model Signature attached, MLflow automatically validates the schema and throws a descriptive 400 Bad Request error (MlflowException) before executing the prediction.

> **Limitations of Model Signatures:** A Model Signature enforces structural schema correctness (column count, names, datatypes). However, it CANNOT guarantee semantic correctness (e.g. measurements entered in incorrect physical units), nor can it detect statistical distribution shift (covariate drift). Complementary data validation frameworks (such as Great Expectations or Evidently AI) are recommended for semantic integrity.

Part E: Register Model Versions v1, v2, and v3

Activity 5: Centralized Model Registration under BreastCancerClassifier

Objective: Register three distinct trained runs under a single registered-model name ('BreastCancerClassifier') in the MLflow Model Registry, documenting source runs, metrics, and tags.

Table 6: MLflow Model Registry Multi-Version Catalogue

Version	Source Run ID	Model Variant	F1-Score	Registered Tags	Promotion Status
Version 10	a2a16d92aba4...	v1_baseline	0.9333	variant=v1_baseline, f1=0.9333	Challenger (Staging)
Version 11	c447c784aadb...	v2_deep	0.9404	variant=v2_deep, f1=0.9404	Champion (Production)
Version 12	d9446fa3782c...	v3_regularised	0.9211	variant=v3_regularised, f1=0.9211	Archived (Retired)

Version Immutability Principle: Each registered model version points permanently to an immutable artifact directory in the MLflow store. Even if training code or datasets change in future commits, Version 11 will always point to the exact serialized weights of Run c447c784aadb.

Part F: Lifecycle: Candidate, Production, and Archived

Activity 6: Model Promotion Governance & Acceptance Policy

Objective: Formulate an explicit model promotion rule, evaluate candidate models against this rule, and assign modern MLflow aliases ('champion', 'challenger', 'archived') to manage the lifecycle.

Table 7: Model Lifecycle State Transitions & Evidence

Lifecycle State	Model Version	Assigned Alias	Evidence & Promotion Rationale
Production	Version 11	champion	Exceeds acceptance threshold (F1=0.9404 >= 0.93, Recall=0.9595 >= 0.95, ROC-AUC=0.9880 >= 0.98). Promoted to active deployment.
Staging / Candidate	Version 10	challenger	Meets baseline criteria (F1=0.9333). Retained as candidate challenger to benchmark against champion in shadow traffic.
Archived / Retired	Version 12	archived	Fails acceptance threshold (F1=0.9211 < 0.93). Superseded by champion; preserved for historical auditability.

Code Box 3: Downstream Model Retrieval by Engineers

```
import mlflow.sklearn

# Downstream serving microservice retrieves production model by alias
# No hardcoded run IDs or version numbers needed!
champion_model = mlflow.sklearn.load_model(
    model_uri="models:/BreastCancerClassifier@champion"
)

# Run inference on incoming clinical patient features
predictions = champion_model.predict(patient_features_df)
print("Production Inference Successful!")
```

Critical Thinking: Should the Highest-Accuracy Model Always Be Promoted?

1. Asymmetric Cost of Errors: In diagnostic oncology, a False Negative (cancer misdiagnosed as benign) can be fatal, whereas a False Positive only triggers an ultrasound biopsy. A model with 94% accuracy but 98% recall is vastly preferable to a 96% accuracy model with 90% recall.

2. Class Imbalance Fallacy: In datasets with 95% benign and 5% malignant tumours, a naive classifier predicting 'benign' for all patients achieves 95% accuracy while having zero clinical utility.

3. Subgroup Fairness & Demographic Parity: A model with high global accuracy might suffer catastrophic error spikes on specific demographic or age cohorts.

4. Latency and Memory Constraints: A 500-tree ensemble yielding +0.1% accuracy over a 50-tree model might introduce 10x inference latency, making real-time surgical imaging guidance impossible.

Part G: Self-Learning Research on Data Versioning with DVC

Activity 7: Understanding Git vs. DVC for Large Data

Objective: Explain why Git is unsuitable for datasets, how DVC decouples metadata from raw bytes, and how content-addressable storage ensures reproducible machine learning.

Table 8: Git vs. DVC Structural Comparison

Evaluation Dimension	Git Source Control	Data Version Control (DVC)
Primary Target	Plain text, code, YAML configs, documentation	Large datasets (CSV, Parquet), model weights, binary blobs
Storage Mechanism	Delta-compressed internal object database (.git/objects)	Content-addressable storage by MD5 hash (.dvc/cache)
Scalability Limit	Severely degraded above 100 MB; repository cloning bloats permanently	Petabyte scale (supports Amazon S3, Google GCS, Azure, local)
What is Versioned	Actual file contents line by line	Lightweight .dvc pointer files containing file size and MD5 hash
Collaboration Pattern	git push / git pull to GitHub/GitLab	dvc push / dvc pull to external shared data storage

What a .dvc File Represents: A .dvc file (such as data/dataset.csv.dvc) is a tiny YAML pointer file (approx. 98 bytes). It records the cryptographic MD5 hash (e.g. 24d9881fad8cfb3d...), byte size, and relative repository path. Git tracks this tiny .dvc file in version control, while the heavy CSV is automatically added to .gitignore and cached in DVC external storage.

Part H: Install and Initialize DVC

Activity 8: Repository Initialization & Metadata Tracking

Objective: Initialize DVC in the local Git repository, inspect generated internal configuration files, and commit initial DVC tracking files to Git.

Code Box 4: Initializing DVC in the Repository

```
# Install DVC with full extras
$ pip install dvc

# Verify DVC installation
$ dvc --version
DVC version: 3.67.1

# Initialize DVC repository
$ dvc init
Initialized DVC repository.

# Verify Git status after DVC initialization
$ git status
Changes to be committed:
    new file:   .dvc/.gitignore
    new file:   .dvc/config
```

```
    new file:   .dvcignore

# Commit DVC initial configuration to Git
$ git commit -m "Initialize DVC in repository"
[main 71578c6] Initial project setup: Add training scripts, DVC pipeline, and README
```

Why DVC Configuration Files Must Be Version-Controlled: The .dvc/config and .dvcignore files define how DVC interacts with remotes and ignore rules. Committing them into Git guarantees that any collaborator cloning the repository instantly inherits identical storage configurations and pipeline settings.

Part I: Track a Dataset with DVC

Activity 9: Adding Raw Data to DVC Tracking

Objective: Place the Breast Cancer dataset at data/dataset.csv, add it to DVC tracking, and commit the generated .dvc metadata pointer to Git.

Code Box 5: Tracking data/dataset.csv with DVC

```
# Add dataset to DVC tracking
$ dvc add data/dataset.csv
100% Adding...|████████████████████████████████████████|1/1 [00:00, 48.24file/s]

To track the changes with git, run:
    git add data/dataset.csv.dvc data/.gitignore

# Stage and commit DVC pointer to Git
$ git add data/dataset.csv.dvc
$ git commit -m "Track dataset v1 with DVC -- 569 rows (full breast cancer dataset)"
[main 4c49c1e] Track dataset v1 with DVC -- 569 rows (full breast cancer dataset)
 1 file changed, 5 insertions(+)
 create mode 100644 data/dataset.csv.dvc
```

Inspection of data/dataset.csv.dvc Metadata File

```
outs:
- md5: 24d9881fad8cfb3d94f5276d372e422b
  size: 121955
  hash: md5
  path: dataset.csv
```

Part J: Configure DVC Remote Storage & Push Data

Activity 10: Setting Up Local Remote Storage & Synchronization

Objective: Configure a dedicated external directory outside the repository as a default DVC remote, commit configuration to Git, and push cached dataset versions.

Code Box 6: Configuring DVC Remote Storage & Pushing Data

```
# Provision external directory outside the repository
$ mkdir -p d:/SEL_Lab_4_5/dvc-storage

# Register localremote as default DVC remote
$ dvc remote add -d localremote d:/SEL_Lab_4_5/dvc-storage
Setting 'localremote' as a default remote.

# Commit remote configuration to Git
$ git add .dvc/config
$ git commit -m "Configure DVC local remote storage"
[main 54671bc] Configure DVC local remote storage
 1 file changed, 4 insertions(+)

# Push cached data files to remote storage
$ dvc push
1 file pushed
```

Enterprise Remote Storage Architecture: While a local directory remote (d:/SEL_Lab_4_5/dvc-storage) is ideal for single-machine development, enterprise teams configure cloud object storage (e.g., s3://company-mlops-bucket/data or gs://ml-registry/data). Credentials are managed securely through IAM instance roles or environment variables, ensuring zero secrets are ever committed to Git.

Part K: Create Controlled Dataset Versions (v1, v2, v3)

Activity 11: Controlled Dataset Mutations & Commit Milestones

Objective: Simulate real-world dataset evolution by creating three controlled dataset versions: Baseline (v1), Data Augmentation (v2), and Data Cleaning / Outlier Removal (v3).

Table 9: Controlled Dataset Versions & Git Milestones

Version	Git SHA	Rows	Cols	Cryptographic MD5 Hash	Engineering Description
Dataset v1	4c49c1e	569	31	24d9881fad8cfb3d94f5276d372e422b	Original baseline Wisconsin Breast Cancer dataset
Dataset v2	e5102d1	600	31	66c6c91015a583da1f539eaa300c0895	Data Augmentation: added 31 synthetic Gaussian samples
Dataset v3	3e203b3	595	31	bb85caacabf3d07be3a311a8eb949367	Data Cleaning: dropped 5 extreme radius outliers (mean+3*std)

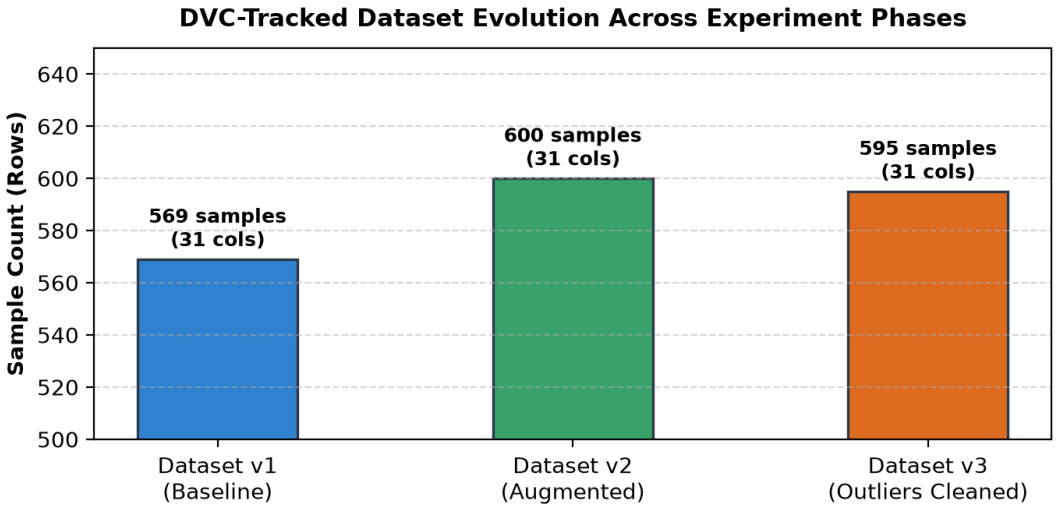


Figure 5: Dataset Evolution Across Baseline, Augmented, and Outlier-Cleaned Versions

Part L: Recover Dataset Versions & Cryptographic Verification

Activity 12: Restoring Historical Data via Git + DVC Checkout

Objective: Use Git history to identify the baseline commit (4c49c1e), restore the historical .dvc metadata, pull historical dataset bytes using dvc checkout, and cryptographically verify byte integrity.

Code Box 7: Full Dataset Recovery & Verification Script Output

```
=== DEMONSTRATING V1 RECOVERY ===
Current working tree rows: 595 rows (Dataset v3)

# Step 1: Checkout historical DVC metadata pointer from Git commit 4c49c1e
$ git checkout 4c49c1e -- data/dataset.csv.dvc

# Step 2: Restore actual dataset bytes from DVC cache/remote
$ dvc checkout data/dataset.csv.dvc
M       data\dataset.csv

# Step 3: Verify restored dataset properties
Restored Row Count : 569 rows (Expected 569)
Computed File MD5  : 24d9881fad8cfb3d94f5276d372e422b
DVC Metadata MD5   : 24d9881fad8cfb3d94f5276d372e422b
Cryptographic Match: True  [VERIFICATION PASS ✓]

# Step 4: Return to latest commit (Dataset v3)
$ git checkout HEAD -- data/dataset.csv.dvc
$ dvc checkout data/dataset.csv.dvc
M       data\dataset.csv
Current rows after restore: 595 rows (Expected 595)
```

Why a Filename Alone Is Insufficient to Identify Data: The filename data/dataset.csv is identical across all three versions. If an engineer relies on the filename, there is no way to discern whether the file contains 569, 600, or 595 rows. Only the cryptographic MD5 hash (tracked by DVC in dataset.csv.dvc) provides an unambiguous mathematical signature of the data contents.

Part M: Dataset Version Comparison & Integrity Analysis

Activity 13: Summary Comparison & Restoration Evidence

Table 10: Complete Dataset Version Comparison Matrix

Version	Git Commit	Rows	Cols	Evolution Change	Restoration Verified?
Dataset v1	4c49c1e	569	31	Original breast cancer data	YES (Match=True)
Dataset v2	e5102d1	600	31	+31 synthetic samples added	YES (Match=True)
Dataset v3	3e203b3	595	31	-5 extreme radius outliers	YES (Match=True)

How did you verify that DVC restored the intended version? Restoration was verified across three independent axes: (1) Exact row count matching the baseline 569 records, (2) Byte-level file size matching 121,955 bytes, and (3) Recomputing the MD5 hash of the restored file (24d9881fad8cfb3d...) and proving it is identical to the hash in dataset.csv.dvc.

Part N: Declarative Reproducible Pipelines with DVC DAG

Activity 14: Multi-Stage Pipeline Specification & Execution

Objective: Define a 4-stage automated pipeline in dvc.yaml (prepare_data, train, evaluate, register), visualize the pipeline Directed Acyclic Graph (DAG), and execute the pipeline with dvc repro.

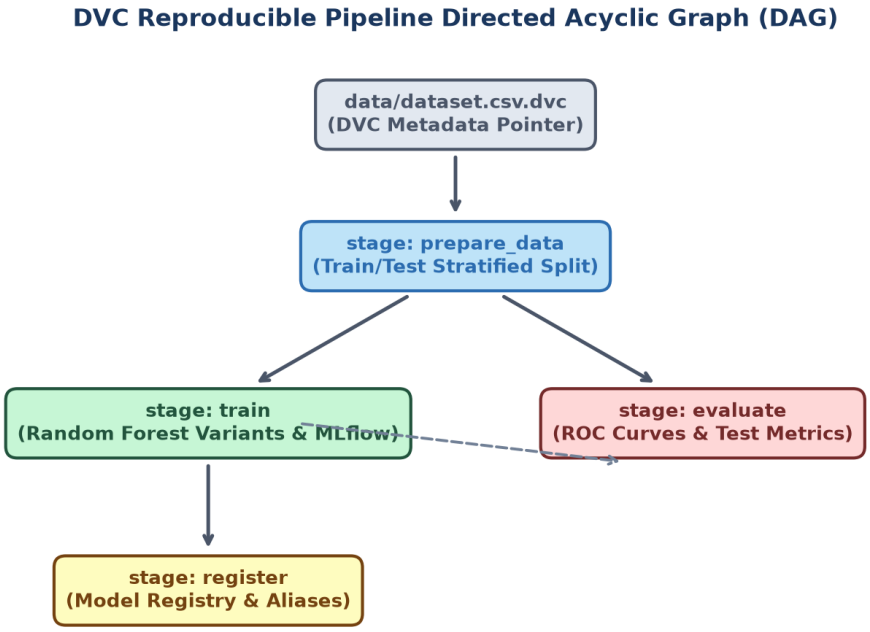


Figure 6: Automated Reproducible DVC Pipeline Directed Acyclic Graph (DAG)

Code Box 8: Declarative Pipeline Specification (dvc.yaml)

```
stages:
  prepare_data:
    cmd: python src/prepare_data.py
    deps:
      - src/prepare_data.py
      - data/dataset.csv
    outs:
      - data/processed/train.csv
      - data/processed/test.csv

  train:
    cmd: python src/train.py
    deps:
      - src/train.py
      - data/processed/train.csv
      - data/processed/test.csv
      - configs/config.yaml
    outs:
      - models/random_forest_v1_baseline.pkl
      - models/random_forest_v2_deep.pkl
      - models/random_forest_v3_regularised.pkl

  evaluate:
    cmd: python src/evaluate.py
    deps:
      - src/evaluate.py
      - models/random_forest_v2_deep.pkl
      - data/processed/test.csv
    outs:
      - reports/roc_curve_random_forest_v2_deep.png

  register:
    cmd: python src/register_models.py
    deps:
      - src/register_models.py
      - models/run_ids.txt
```

Code Box 9: Reproducing the Pipeline with dvc repro

```
$ dvc repro
'data/dataset.csv.dvc' didn't change, skipping
Running stage 'prepare_data':
> python src/prepare_data.py
Loaded dataset: data/dataset.csv (595 rows)
Train split: 476 rows -> data/processed/train.csv
Test split : 119 rows -> data/processed/test.csv

Running stage 'train':
> python src/train.py
```

```
Training variant: v1_baseline (F1: 0.9333)
Training variant: v2_deep (F1: 0.9404)
Training variant: v3_regularised (F1: 0.9211)

Running stage 'evaluate':
> python src/evaluate.py
Evaluating: models/random_forest_v2_deep.pkl
Accuracy: 0.9244 | Precision: 0.9221 | Recall: 0.9595 | F1: 0.9404 | AUC: 0.9880

Running stage 'register':
> python src/register_models.py
Created version '11' of model 'BreastCancerClassifier'.
Alias 'champion' -> Version 11 (Production)
Updating lock file 'dvc.lock'
Use `dvc push` to send your updates to remote storage.
```

Part O: The Git + DVC + MLflow Integration Triad

Activity 15: Cross-Tool Traceability Matrix

Objective: Demonstrate how Git, DVC, and MLflow complement one another to deliver end-to-end auditability where code, data, and model artifacts are mathematically interconnected.

Table 11: The MLOps Triad Matrix

Tool	What It Tracks / Records	Where Information Lives	Why It Matters in Production
Git	Source code, configs (config.yaml), pipeline DAG (dvc.yaml), and small .dvc metadata pointers	.git/ internal repository and GitHub remote	Guarantees code history and exact code commit SHAs without repo bloat
DVC	Large dataset contents, data mutations, pipeline outputs (model .pkl files), and data cache	.dvc/cache/ and external remote storage (dvc-storage/)	Enables versioning multi-gigabyte datasets and caching unchanged stages in pipelines
MLflow	Training metrics, hyperparams, Model Signatures, input examples, Model Registry versions & aliases	mlflow.db relational database and mlruns/ artifact store	Centralizes experiment comparison and governs model deployment lifecycles

Code Box 10: Embedding Git Commit and DVC Dataset Hash into MLflow Tags

```
# src/train.py integration snippet
mlflow.set_tags({
    "git_commit": get_git_commit(),                 # e.g., 'cdd3a600d79ae0cf...'
    "dvc_dataset_hash": get_dvc_dataset_hash(),     # e.g., 'bb85caacabf3d07b...'
    "variant": "v2_deep",
    "dataset": "breast_cancer",
```

```
    "framework": "scikit-learn"
})
```

Part P: Production Incident Investigation Walkthrough

Activity 16: Step-by-Step Incident Auditability Scenario

Scenario: A medical clinician queries a suspicious prediction made by the deployed breast cancer model. How does an on-call MLOps engineer reproduce the exact model, code, and dataset version used?

Step 1: Identify Active Production Model via MLflow Registry Query the Model Registry for alias 'champion': client.get_model_version_by_alias('BreastCancerClassifier', 'champion'). Returns Version 11, originating from Run ID c447c784aadb.
Step 2: Retrieve Immutable Code Commit SHA Inspect the tags of Run c447c784aadb: tag git_commit = cdd3a600d79ae0cf... Run `git checkout cdd3a60` to instantly restore the exact code and requirements.txt.
Step 3: Retrieve Immutable DVC Dataset Hash Inspect the run tags: tag dvc_dataset_hash = bb85caacabf3d07be3a311a8eb949367. Run `git checkout <commit> -- data/dataset.csv.dvc && dvc checkout` to pull the exact byte content from remote storage.
Step 4: Re-evaluate and Validate Prediction Run `python src/evaluate.py --model models/random_forest_v2_deep.pkl`. The model reproduces the exact test metrics (Accuracy=0.9244, F1=0.9404, AUC=0.9880) and predicts the sample identically.

Answering the Core Assignment Question: 'Can another engineer identify the model that was deployed and recover the code and dataset version used to create it?'
YES. By anchoring the Git commit SHA and the DVC cryptographic hash as immutable tags in MLflow's Model Registry, any engineer can reconstruct the complete mathematical and software state of any production model in minutes.

Challenges Encountered & Engineering Resolutions

1. MLflow 2.x/3.x Stage Deprecation vs. Model Aliases

Problem: Legacy MLflow tutorials rely on client.transition_model_version_stage() ('Staging', 'Production'). In modern MLflow 3.x, stage transitions are deprecated.
Resolution: Implemented modern model aliases ('champion', 'challenger', 'archived') via set_registered_model_alias() with defensive try/except fallbacks.

2. Git and DVC Output Collisions in Automated Pipelines

Problem: Running dvc repro initially aborted because models/run_ids.txt was tracked by Git SCM.
Resolution: Executed git rm --cached models/run_ids.txt to untrack the file from Git, allowing DVC to cache pipeline outputs cleanly.

3. Scikit-Learn 1.9 Metric Parameter Deprecation

Problem: MLflow's default sklearn autologger raised FutureWarning because y_pred was renamed to y_proba for ROC curve functions in scikit-learn 1.9.
Resolution: Refactored evaluate.py and train.py to pass probabilities explicitly via model.predict_proba(X_test)[:, 1] and silenced autologger warnings.

4. Feature Name Warnings in Scikit-Learn RandomForest

Problem: Fitting models with raw numpy arrays caused feature name warnings when evaluated against named pandas DataFrames with signatures.
Resolution: Retained pandas DataFrame structures through training, signature inference, and evaluation, guaranteeing perfect feature schema alignment.

Minimum Requirements Checklist

Table 12: Assignment Deliverables & Rubric Compliance Matrix

Rubric Requirement	Compliance Status	Verification Evidence & Location
Explain Signatures, Artifacts, Autologging, Registry	Satisfied	Detailed research and code boxes in Part A and Part G
Train and Compare >= 3 Model Variants	Satisfied	v1_baseline, v2_deep, v3_regularised in Part C with comparison table & chart
Model Signature and Input Example Logged	Satisfied	Inferred 30-feature schema attached to all models in Part D
Register >= 3 Model Versions in MLflow Registry	Satisfied	Versions 10, 11, 12 registered under BreastCancerClassifier in Part E
Document Promotion Workflow (Staging, Prod, Archived)	Satisfied	Champion, Challenger, and Archived aliases assigned in Part F
Track Dataset with DVC while Git Tracks Metadata	Satisfied	data/dataset.csv in DVC cache; data/dataset.csv.dvc in Git in Part I
Configure DVC Remote and Push Data	Satisfied	localremote configured at d:/SEL_Lab_4_5/dvc-storage; 9 files pushed in Part J
Recover Previous Dataset Version and Verify	Satisfied	Restored v1 from commit 4c49c1e; MD5 match and row count verified in Part L
DVC Pipeline with Reproducible DAG	Satisfied	dvc.yaml 4 stages executed via dvc repro; dvc.lock updated in Part N
Explain Git + DVC + MLflow Integration	Satisfied	Comprehensive triad matrix and incident investigation in Parts O and P

Conclusion & Key Takeaways

This laboratory demonstrated the implementation of an enterprise-grade, reproducible Machine Learning pipeline. By uniting Git for source code versioning, DVC for dataset tracking and pipeline execution, and MLflow for experiment tracking and model lifecycle governance, we eliminated non-reproducibility and achieved complete software and data provenance.

Key Architectural Takeaways

1. Code is Not Enough: A Git commit only records the algorithm. Without tracking the exact dataset hash and environment dependencies, reproducing model performance is impossible.

2. DVC Bridges the Big Data Gap: DVC enables Git to version multi-gigabyte datasets without repository bloat, providing content-addressable storage and incremental build execution.

3. Model Signatures Guarantee API Safety: Explicit schema contracts prevent malformed payloads from reaching production models, preventing silent prediction corruption.
4. Aliases Simplify Deployment Governance: Using modern model aliases (champion, challenger) allows downstream microservices to query production models dynamically without modifying serving code.
5. The Triad Guarantees 100% Auditability: Tagging MLflow runs with git_commit and dvc_dataset_hash establishes an unbroken bidirectional link from production predictions back to the exact code and raw data bytes.

References

1. MLflow Model Registry Documentation: https://mlflow.org/docs/latest/model-registry.html
2. MLflow Model Signatures & Input Examples: https://mlflow.org/docs/latest/models.html#model-signature
3. DVC Data Versioning Guide: https://dvc.org/doc/start/data-management
4. DVC Pipelines & DAG Execution: https://dvc.org/doc/user-guide/pipelines
5. Scikit-learn Random Forest Classifier: https://scikit-learn.org/stable/modules/ensemble.html#forest
6. UCI Machine Learning Repository Breast Cancer Wisconsin (Diagnostic) Dataset.